

REPRODUCING THE CRITICAL TRAINING SET SIZE OF AN EFFECTIVE THEORY OF GROKING

Charlotte Natalie Licht

Group 73 · Replication Track
Advanced Topics in Deep Learning, University of Copenhagen
pcb887@alumni.ku.dk

Code: <https://github.com/lichtnatalie/atdl-a2-grokking-replication>

ABSTRACT

Liu et al. explain grokking with an effective theory that predicts a critical training data fraction $r_c \approx 0.4$ for a toy addition task. I test this prediction with the authors’ released code. The training-free part reproduces. The probability that the training data determine the linear representation crosses one half at $r = 0.41$, and my implementation agrees with the authors’ on all 1800 training sets checked. Trained networks, however, do not follow the prediction with the released settings. Over twenty seeds per training size, half of the runs succeed within 10^4 steps only at $r \approx 0.76$. Only 70 of the 138 runs whose data determine the representation actually find it. Decoder weight decay has almost no effect. The decoder learning rate matters a lot, since a slower decoder raises this number to 96 and a faster one lowers it to 15. Whether a network finds the structure thus depends on how fast the decoder learns, which the theory does not model.

Index Terms— Grokking, reproducibility, representation learning, effective theory, critical training set size

1. INTRODUCTION

Neural networks sometimes generalise long after they have fit their training data, a behaviour known as *grokking* [1]. It was first seen on small algorithmic tasks, where a network reaches perfect training accuracy and then suddenly starts to get unseen examples right, thousands of steps later. This does not fit the normal picture of overfitting, in which test performance only gets worse once the training set has been memorised.

Liu et al. [2] explain grokking as representation learning, so the network generalises once its embeddings become structured. For a toy addition task they also make a quantitative prediction. Below a critical training data fraction $r_c \approx 0.4$, the training data usually do not determine the right structure, and the paper links this to no generalisation. Above, training sets increasingly determine the structure. I chose to test this prediction because it is the sharpest claim in the paper, because each run takes minutes on a laptop, and because the authors released their code.

This report follows the Replication Track [3]. I rerun the experiment behind the paper’s Fig. 4 with the authors’ own code, first exactly as released and then with more seeds, and I add two ablations on the decoder.

The theory’s training-free prediction reproduces. However, the trained networks do not follow it with the released settings. Half of the trained networks find the structure only at $r \approx 0.76$, and in my runs having enough information in the data was necessary but not sufficient. My ablations suggest why. The theory predicts when the data *determine* the representation, but whether training actually *finds* it depends on how fast the decoder learns.

2. BACKGROUND

Liu et al. [2] study grokking on a task whose correct solution is known in advance, so that one can check directly whether a network has found it. Their argument has two stages summarised in Fig. 1. First, the training data must contain enough information to determine the solution, and second, the training must then actually find it.

2.1. Task and model

The task needs a known correct solution, so Liu et al. use non-modular addition on the symbols $\{0, \dots, p-1\}$, here with $p = 10$. Because addition is a commutative operation, (a, b) and (b, a) are treated as one sample, so the full dataset D_0 contains $|D_0| = p(p+1)/2 = 55$ pairs. A training set $D \subset D_0$ is drawn at random, the remaining pairs are held out, and $r = |D|/|D_0|$ is the *training data fraction*.

The model gives each symbol a trainable embedding $\mathbf{E}_a \in \mathbb{R}^{d_{\text{in}}}$, adds the two embeddings of a pair, and passes the sum to a decoder network:

$$(a, b) \mapsto \text{Dec}(\mathbf{E}_a + \mathbf{E}_b). \quad (1)$$

Each possible sum c has a fixed random target vector $\mathbf{Y}_c$, and the model predicts the sum of those target is closest to its output. Since the addition of embeddings is built into the

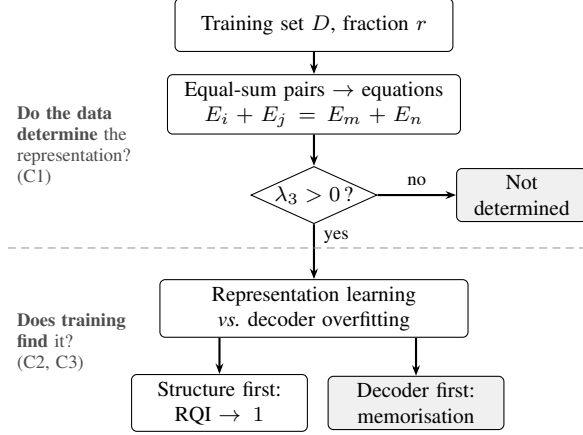


Fig. 1. The argument tested in this study. Upper part, whether the training set determines the linear representation depends only on the equal-sum pairs it contains, summarised by $\lambda_3 > 0$ (claim C1). Lower part, whether training then finds that representation depends on the competition between representation learning and decoder overfitting (claims C2 and C3).

model, the only way it can answer an unseen pair is through the arrangement of its embeddings. As in the paper’s Fig. 4, I use one-dimensional embeddings ($d_{\text{in}} = 1$) and write $\mathbf{R} = [\mathbf{E}_0, \dots, \mathbf{E}_{p-1}]$ for the representation.

2.2. Representation quality

To measure whether the embeddings have the right arrangement, Liu et al. use pairs with equal sums. Say $i + j = m + n$ and the model maps both pairs to the same point, $\mathbf{E}_i + \mathbf{E}_j = \mathbf{E}_m + \mathbf{E}_n$. Then the decoder cannot tell the two pairs apart, so a correct answer for one transfers to the other. Such a quadruple is called a *parallelogram*.

For a model with zero training loss and an injective decoder, every two training pairs with the same sum must form a parallelogram ([2], Prop. 2). These are

$$P_0(D) = \left\{ (i, j, m, n) \mid \begin{array}{l} (i, j) \in D, (m, n) \in D, \\ i + j = m + n \end{array} \right\}. \quad (2)$$

The *representation quality index* is the fraction of all parallelograms of the full dataset, $P_0 \equiv P_0(D_0)$, that the representation realises within a tolerance δ . Writing $P(\mathbf{R}, \delta)$ for the parallelograms in P_0 with $|\mathbf{E}_i + \mathbf{E}_j - \mathbf{E}_m - \mathbf{E}_n| \leq \delta$,

$$\text{RQI}(\mathbf{R}) = \frac{|P(\mathbf{R}, \delta)|}{|P_0|} \in [0, 1]. \quad (3)$$

Evenly spaced embeddings, $\mathbf{E}_k = \mathbf{a} + k\mathbf{b}$, realise every parallelogram and give $\text{RQI} = 1$. This is the *linear representation*, where random embeddings give $\text{RQI} \approx 0$.

2.3. Effective theory and the grokking rate

The effective theory is about whether a training set contains enough parallelograms to force the linear representation and, it is, then how quickly training gets there. To make this tractable, Liu et al. leave out the decoder, so the normalised embeddings descend a loss on violated parallelograms.

In one dimension each parallelogram in $P_0(D)$ is a linear equation $E_i + E_j = E_m + E_n$, and these equations form the set $A(P_0(D))$. The effective loss $\ell_{\text{eff}} = \ell_0/Z_0$ is the mean squared violation ℓ_0 of the equations, normalised by $Z_0 = \sum_k |\tilde{\mathbf{E}}_k|^2$. And, because ℓ_0 is quadratic, gradient descent on it is linear, $d\mathbf{R}/dt = -\mathbf{H}\mathbf{R}$, with the Hessian

$$\mathbf{H}_{ij} = \frac{1}{Z_0} \frac{\partial^2 \ell_0}{\partial \mathbf{E}_i \partial \mathbf{E}_j}. \quad (4)$$

Each eigenvector of $\mathbf{H}$ is a direction in which the embeddings can move, and its eigenvalue, sorted as $\lambda_1 \leq \dots \leq \lambda_p$, is the rate at which deviations in that direction die out. A zero eigenvalue means that the training equations do not constrain that direction.

Translating or scaling all embeddings would not break a parallelogram, so $\lambda_1 = \lambda_2 = 0$ for every training set. If $\lambda_3 = 0$, a non-linear representation fits the training equations equally well. If $\lambda_3 > 0$, every other direction is constrained, the nullity n_0 of $A(P_0(D))$ is two, and the linear representation is unique up to these two changes.

In that case, λ_3 is the slowest rate that matters, so it sets the timescale for approaching the linear representation. Liu et al. call it the *grokking rate* and give the time towards the linear structure as

$$t_h = \frac{1}{\lambda_3}, \quad n_h = \frac{1}{\lambda_3 \eta}, \quad (5)$$

where η is the embedding learning rate and n_h the number of steps.

2.4. Claims under test

I test three claims of [2] for $p = 10$:

C1 ([2], Fig. 4a.) The probability that a random training set gives $\lambda_3 > 0$ rises sharply at a critical data fraction $r_c \approx 0.4$.

C2 ([2], Fig. 4b.) “The number of steps to reach $\text{RQI} > 0.95$ is seen to have a phase transition at $r_c = 0.4$, agreeing with the proposed effective theory.”

C3 ([2], Fig. 5b.) Among trained networks, the steps to $\text{RQI} > 0.95$ decrease with λ_3 , as predicted by $t \sim 1/\lambda_3$.

Claim 1 (C1) depends only on the training set, the upper part of Fig. 1. Claim 2 (C2) and Claim 3 (C3) depend on what training does with it, the lower part.

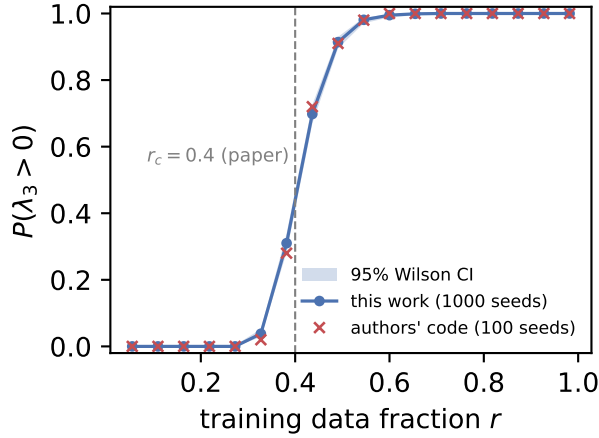


Fig. 2. C1. Probability that a random training set determines the linear representation ($\lambda_3 > 0$) against the training data fraction. 1000 training sets per size (band: 95% Wilson interval). Crosses: the authors’ code on 100 of the same training sets. Dashed: $r_c = 0.4$ as stated in the paper.

3. REPRODUCTION PROTOCOL

I begin by rerunning the authors’ experiment exactly as released, to check that their code reproduces their own saved results. Because C2 claims a transition in the probability that a network finds the linear representation, I then repeat the experiment with enough runs to estimate that probability at each training data fraction.

3.1. Code and configuration

All runs use the authors’ released code [4] without any modification, so that a discrepancy cannot come from a reimplementation. The code is pinned at commit 0229df9 (November 2022) and run on CPU with PyTorch 2.14.

The paper does not state the hyperparameters behind its own Fig. 4(b), and its appendix gives only the decoder architecture and the embedding learning rate. Instead, I use the settings of the released script that generates this figure, listed in Table 1. The script trains three seeds per training size. I train twenty, including the authors’ three. One seed fixes the targets, the training split, the initialisation and the order of minibatches, giving 220 runs per configuration.

3.2. Measurements

For each run, the question is to see whether and when it finds the linear representation, and how this compares with what the theory predicts for its training set.

Steps to RQI > 0.95. RQI is computed at every step as in the released code, on normalised embeddings and with

Table 1. Training configuration of the released script, used for all runs. Ablations change one row at a time (marked *).

Decoder	MLP 1–200–200–30, tanh
Embeddings	$d_{\text{in}} = 1$, initialised $\mathcal{U}(-\frac{1}{2}, \frac{1}{2})$
Targets $\mathbf{Y}_c$	$\mathcal{N}(\mathbf{0}, \mathbf{I})$ in $\mathbb{R}^{30}$, squared-error loss
Optimiser	AdamW [5] for both parts
Learning rates	embeddings 10^{-3} , decoder* 10^{-4}
Weight decay	none*
Minibatch	$\min(45, D)$ training pairs
Budget	10^4 steps
Training sizes	$ D \in \{5, 10, \dots, 50, 54\}$ ($r = 0.09\text{--}0.98$)
Seeds	0–19 per size (released script: 0–2)

tolerance $\delta = 0.1$. As in the paper, I record the number of steps until RQI > 0.95. At each training size I also record the fraction of runs that get there within 10^4 steps, and where this fraction passes one half.

Runs that do not succeed. The released code reports the last step for these runs, which cannot be told apart from a run that succeeds exactly at the end. I therefore have chosen to record success separately, treat unsuccessful runs as infinitely slow when taking medians, and leave the median undefined if more than half the runs at a size fail.

Theory. I computed λ_3 with my own implementation of (4), independent of the authors’ code. For C1, $\Pr(\lambda_3 > 0)$ is estimated from 1000 random training sets per training size, and for C3, λ_3 is computed for the training set of every trained run.

I set $Z_0 = p$, its value for unit-variance embeddings. As a result, my λ_3 differs from the paper’s by a constant factor, which is why it is compared only through ranks and slopes.

3.3. Statistical analysis

Every configuration is trained on the same 220 training sets. Runs with the same seed are not independent, since across sizes they share targets, initialisation, and nested training sets. Configurations are therefore compared seed by seed, and I count the seeds whose number of successes with $\lambda_3 > 0$ goes up or down, and use an exact sign test.

Within one training size the seeds are independent, so success probabilities there are given with Wilson 95% intervals [6]. For C3, Spearman’s ρ and the log–log slope (theory: -1) are only descriptive, because λ_3 also grows with the training size.

3.4. Ablations

The ablations test which part of training decides whether a determined representation is actually found. Each repeats all 220 runs with one setting changed, either (A1) decoder weight decay of 1 or 5, or (A2) a decoder learning rate of 10^{-5} or 10^{-3} . Both are axes of the paper’s own phase diagrams, which describe a competition between representation

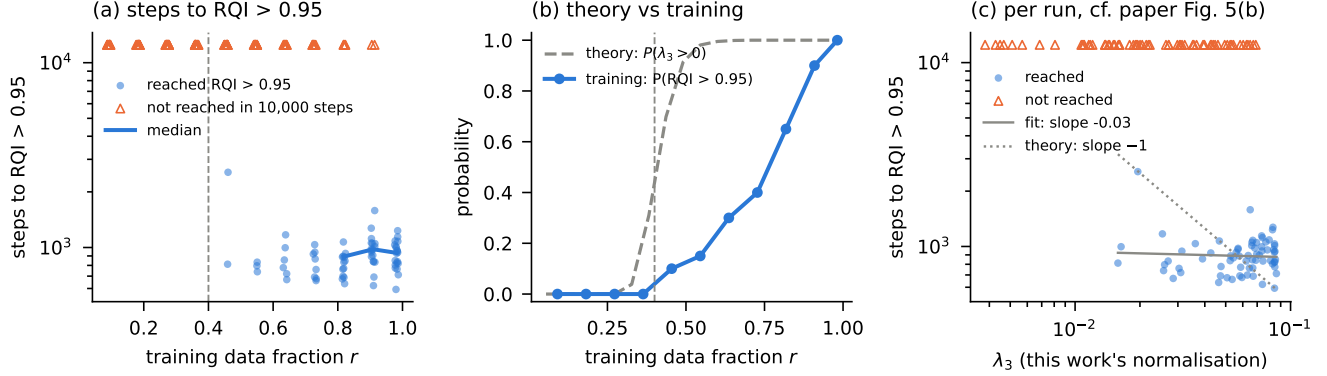


Fig. 3. C2 and C3 with the released configuration, 20 seeds per size. (a) Steps to RQI > 0.95. Triangles: runs that never reach it in 10^4 steps. Line: median with failed runs counted as ∞ . (b) Fraction of runs reaching RQI > 0.95, against the theory’s $\Pr(\lambda_3 > 0)$. (c) Steps against λ_3 for every run with $\lambda_3 > 0$ (compare the paper’s Fig. 5b). Grey: least-squares fit to the successful runs. Dotted: the slope -1 predicted by the theory.

learning and the decoder. I also retrain samples of unsuccessful runs with $\lambda_3 > 0$ for 5×10^4 steps, to tell slow runs from stuck ones.

3.5. Deviations from the original study

I use the paper’s model, task and training procedure. No smaller or modified model is used. The study deviates in four documented ways, here twenty seeds instead of three, the released configuration in place of the undocumented settings of the published Fig. 4(b), λ_3 known up to a constant factor, and handling of runs that never succeed.

4. RESULTS

4.1. C1: the training-free prediction

C1 reproduces and Fig. 2 shows the fraction of random training sets with $\lambda_3 > 0$. It is zero up to $r \approx 0.27$, then rises steep and passes 0.5 at $r = 0.41$, very close to the $r_c \approx 0.4$ reported in the paper. My own implementation and the authors’ code agree on all 1800 training sets that I checked with both. Among training sets with $\lambda_3 > 0$, the median λ_3 also grows steadily with r , so according to the theory more data should mean faster learning.

4.2. C2: steps to RQI > 0.95

C2 does not reproduce with the released configuration. I first checked that I am running the same experiment as the authors. For their three seeds, the steps to RQI > 0.95 in my runs match the values saved in their repository in all 33 runs, so the setup reproduces their runs exactly.

With twenty seeds per size the picture changes (Fig. 3a,b). No run finds the structure below $r = 0.45$. When moving

above that success becomes more likely only slowly, and 10% of runs succeed at $r = 0.45$, 40% at $r = 0.73$, and all of them only at $r = 0.98$. The fraction of successful runs passes half only at $r \approx 0.76$, while the theory’s curve does so at $r = 0.41$ (Fig. 3b).

None of the 82 runs with $\lambda_3 = 0$ found the structure, which is consistent with the theory. But only 70 of the 138 runs with $\lambda_3 > 0$ did. In my runs, having enough parallelograms in the training set was necessary, but not sufficient.

The failed runs do not seem to be just slow. I retrained ten of them, all with $\lambda_3 > 0$, for 5×10^4 steps. None reached the threshold, and in nine of them RQI did not change at the saved checkpoints after step 5000 (final values 0.06-0.57).

My runs also look different from the published Fig. 4(b). Here, failures stop around $r \approx 0.55$, and successful runs need about 150-550 steps, fewer with more data. Most of mine need between about 650 and 1200 steps, whatever the value of r . The authors’ own saved results for the released script look like mine, not like the published panel. Since the paper does not say which settings were used for that panel, the reason for the difference is unclear.

4.3. C3: dependence on the grokking rate

C3 does not reproduce either. The claim is that the time towards the linear structure scales like $1/\lambda_3$, so runs with a larger λ_3 should succeed sooner. Among the 70 successful runs there is no such trend (Fig. 3c). The Spearman correlation between steps and λ_3 is 0.15, and the log-log slope is -0.03 instead of -1 . The paper’s Fig. 5(b) also shows runs with $\lambda_3 > 0$ that fail within 10^4 steps, which its text does not discuss.

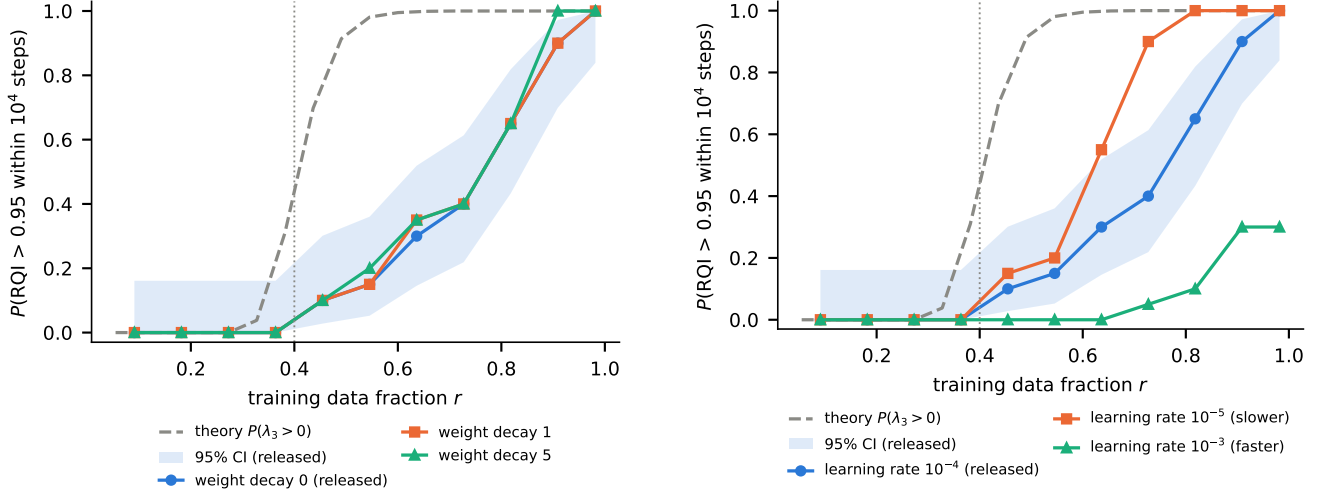


Fig. 4. Fraction of runs reaching RQI > 0.95 within 10^4 steps under the two ablations, 220 runs per setting on the same training sets. The dashed line shows the theory’s $\Pr(\lambda_3 > 0)$. To the left is the decoder weight decay (A1). And to the right is the decoder learning rate (A2).

Table 2. All configurations, 220 runs each on the same training sets. Successes, where runs reaching RQI > 0.95 within 10^4 steps, split by whether the run’s training set has $\lambda_3 > 0$. r at 50% are training data fraction at which half of all runs succeed. $+/-$ are number of the 20 seeds whose successes with $\lambda_3 > 0$ go up/down relative to the released configuration (decoder learning rate 10^{-4} , no weight decay). p are exact sign test over seeds.

Decoder setting	Successes		Seeds vs. released		
	$\lambda_3 > 0$	$\lambda_3 = 0$	r at 50%	$+/-$	p
Released	70/138	0/82	0.76	-	-
Weight decay 1	71/138	0/82	0.76	1/0	1.0
Weight decay 5	74/138	0/82	0.76	4/0	0.13
Learning rate 10^{-5}	96/138	0/82	0.62	14/2	0.004
Learning rate 10^{-3}	15/138	0/82	-	0/20	2×10^{-6}

5. ABLATIONS

Since $\lambda_3 > 0$ is not enough, something in training must decide whether a run finds the structure. The paper describes a “competition between representation learning and decoder overfitting” (lower half of Fig. 1). Decoder weight decay and the decoder learning rate are two of the settings the paper’s phase diagrams use to control this competition, so I varied them. Table 2 summarises the results, and Fig. 4 shows them per training size.

5.1. A1: decoder weight decay

Weight decay does not matter much. With decay 1 or 5, 71 and 74 of the 138 runs with $\lambda_3 > 0$ succeed, compared with 70 without it. The point where half the runs succeed stays at $r \approx 0.76$. With a decoder learning rate of 10^{-4} , a weight decay of 5 in AdamW would on its own shrink the decoder weights to under 1% of their size over 10^4 steps. Even so, only four of the 68 failed runs recover.

5.2. A2: decoder learning rate

The decoder learning rate has a large effect, in the direction the paper’s phase diagrams suggest. Setting the decoder learning rate to 10^{-5} raises the number of successful runs with $\lambda_3 > 0$ from 70 to 96, and moves the 50% point from $r \approx 0.76$ to $r \approx 0.62$. On the same training sets, 28 runs go from failure to success and only two the other way, and 14 of the 20 seeds improve. A faster decoder (10^{-3}) does the opposite. Only 15 runs succeed, and the success probability never gets above 0.30. Its 123 failed runs with $\lambda_3 > 0$ fit the training set (at least 93%) but get only 25% of the held-out pairs right.

With the slow decoder, successful runs need fewer steps as r grows, where the median is about 3500 at $r = 0.64$ and 2500 at $r = 0.98$, as the paper reports. Within one training size, steps and λ_3 show no consistent relation. The slow decoder does not close the gap completely, since 42 runs with $\lambda_3 > 0$ still fail. I retrained eight of them for 5×10^4 steps. One passed the threshold at step 21,217 and then fell back to 0.67, and the other seven stayed below RQI 0.4.

6. DISCUSSION AND CONCLUSION

The first part of the paper’s argument holds up well. Whether a training set pins down the linear representation can be computed exactly, and both my implementation and the authors’ give a transition at $r \approx 0.4$. The second part holds up less well. With the released settings, trained networks find the structure less reliably than the theory suggests, and the time they need does not follow $1/\lambda_3$.

In terms of Fig. 1, the effective theory also predicts the lower half, but only for an gradient flow without a decoder. It says when the data *determine* the representation, and not if a trained network *finds* it. One explanation that fits my results is that the decoder sometimes overfits the training set before the embeddings have become structured. Once that happened, the training loss is close to zero. This would explain why most failed runs stop improving. The effective loss has no decoder, so it cannot describe this. Slowing the decoder down helps and speeding it up makes it worse, which matches the competition between representation learning and decoder overfitting that the paper describes in its Sec. 4. But I did not measure gradients directly.

My results show that C1 is correct, and the published Fig. 4(b) may come from settings closer to the slow-decoder runs. They also show that the agreement in that figure depends on training settings the paper does not report, and that $\lambda_3 > 0$ alone does not predict whether a network will find the structure.

The study has some limitations. I only looked at addition with $p = 10$ and one-dimensional embeddings, and since training uses AdamW rather than gradient flow, only trends can be compared with the theory. With twenty seeds per size, the success probabilities at intermediate fractions are still uncertain by about ± 0.2 .

For reproducibility, it would have helped if the paper had stated the settings and the number of seeds behind each figure. Reporting runs that never succeed as a separate group would also have helped. Both turned out to matter for me to understand what the published figures show.

7. COMPLIANCE WITH ETHICAL STANDARDS

This is a numerical simulation study on synthetic data, for which no ethical approval was required.

8. REFERENCES

- [1] Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra, “Grokking: Generalization beyond overfitting on small algorithmic datasets,” *arXiv preprint arXiv:2201.02177*, 2022.
- [2] Ziming Liu, Ouail Kitouni, Niklas Nolte, Eric J. Michaud, Max Tegmark, and Mike Williams, “Towards understanding grokking: An effective theory of representation learning,” in *Advances in Neural Information Processing Systems*, 2022, vol. 35, pp. 34651–34663.
- [3] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d’Alché Buc, Emily Fox, and Hugo Larochelle, “Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program),” *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.
- [4] Ziming Liu, Niklas Nolte, Ouail Kitouni, and Eric J. Michaud, “grokking-squared: Code for “towards understanding grokking”,” <https://github.com/ejmichaud/grokking-squared>, 2022, Commit 0229df9, accessed 1 October 2026.
- [5] Ilya Loshchilov and Frank Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations*, 2019.
- [6] Edwin B. Wilson, “Probable inference, the law of succession, and statistical inference,” *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.